COL334 Assignment 3: Routing, TCP/IPv4 over Raw Sockets and HTTP

Report

TODO Name 1 (TODO Entry No. 1)
TODO Name 2 (TODO Entry No. 2)

Setup notes. Four Ubuntu VMs (client, r1, r2, server) with four isolated internal networks (LAN A, LAN B, LAN C, Link D). Each machine is configured only by its `netcfg/*.sh` script, run once as root after setting `rp_filter=1` on every interface. Before running our programs on the VMs we add (by hand, *not* in `netcfg` or in our code, as allowed on Piazza) an iptables rule that drops the RSTs the kernel would otherwise send for our raw-socket connections:

```
client$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST --sport 61000:65535 -j DROP
server$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST --sport 8080:8083 -j DROP
```

Our server is therefore always started with a port in 8080–8083.

R1. Adapters, addresses and routes

Each VM's adapters are attached only to internal networks: client = LAN A + Link D, r1 = LAN A + LAN B, r2 = LAN B + LAN C + Link D, server = LAN C (TODO: fill in the internal-network names used in the hypervisor, and mention the extra SSH adapter if you kept one). Table 1 shows what the scripts configure; `IF_X` is the interface name passed to the script for network X.

Machine	Interface	Address	Routes installed by the script (besides connected ones)
client	IF_A, IF_D	10.10.1.10/24, 10.10.4.2/30	`default via 10.10.1.1 dev IF_A`
r1	IF_A, IF_B	10.10.1.1/24, 10.10.2.1/24	`default via 10.10.2.2 dev IF_B` (the single route, A5)
r2	IF_B, IF_C, IF_D	10.10.2.2/24, 10.10.3.1/24, 10.10.4.1/30	`10.10.1.10/32 via 10.10.4.2 dev IF_D`
			`10.10.1.0/24 via 10.10.2.1 dev IF_B` (no default, A6)
server	IF_C	10.10.3.10/24	`default via 10.10.3.1 dev IF_C`

Table 1: Addressing and static routes. `ip_forward` is 1 on r1/r2 and 0 on the client and server (A4).

Why these routes: the client has only a default route via r1, so everything it originates goes client→r1→r2→server (A2) with source 10.10.1.10. r1 needs just one route (default via r2) because LAN C and Link D are both behind r2. On r2, the `/32` host route for the client beats the `/24` by longest-prefix match, so everything addressed to 10.10.1.10 (server replies in particular) leaves over Link D (A3), while the rest of LAN A (i.e. r1's 10.10.1.1) is still reached through r1 (A1). The server needs only a default route via r2.

SCREENSHOT TODO:
`r1_adapters_client.png`

Hypervisor network-adapter settings of the client VM (every adapter tab, or `VBoxManage showvminfo client | grep NIC` on the host).

SCREENSHOT TODO:
`r1_ipaddr_route_client.png`

On client: `ip -br addr; ip route`

Figure 1: Client: adapter settings (left), `ip -br addr` and `ip route` (right).

SCREENSHOT TODO: `r1_adapters_r1.png`

Adapter settings of the r1 VM (both adapters).

SCREENSHOT TODO:
`r1_ipaddr_route_r1.png`

On r1: `ip -br addr; ip route`

Figure 2: r1: adapter settings (left), `ip -br addr` and `ip route` (right).

SCREENSHOT TODO: `r1_adapters_r2.png`

Adapter settings of the r2 VM (all three adapters).

SCREENSHOT TODO:
`r1_ipaddr_route_r2.png`

On r2: `ip -br addr; ip route`

Figure 3: r2: adapter settings (left), `ip -br addr` and `ip route` (right).

SCREENSHOT TODO:
`r1_adapters_server.png`

Adapter settings of the server VM.

SCREENSHOT TODO:
`r1_ipaddr_route_server.png`

On server: `ip -br addr; ip route`

Figure 4: server: adapter settings (left), `ip -br addr` and `ip route` (right).

R2. traceroute in both directions

Expected output (`traceroute -n`, default UDP probes):

```
client$ traceroute -n 10.10.3.10          server$ traceroute -n 10.10.1.10
 1  10.10.1.1   (r1, LAN A)                1  10.10.3.1   (r2, LAN C)
 2  10.10.4.1   (r2, Link D)               2  10.10.1.10  (client)
 3  10.10.3.10  (server)
```

Client → server. *Hop 1:* the TTL=1 probe goes to r1 (the client's default gateway). r1 sends ICMP Time Exceeded back out of LAN A, so its source is r1's LAN A address 10.10.1.1. *Hop 2:* the TTL=2 probe is forwarded by r1 to r2 over LAN B, where it expires. r2 builds an ICMP Time Exceeded addressed to 10.10.1.10. Linux picks the source address of an ICMP error from the route used to send it (`icmp_errors_use_inbound_ifaddr=0`), not from the interface the probe arrived on. r2's route to 10.10.1.10 is the `/32` via Link D, so the reply leaves on IF_D with source 10.10.4.1. That is why r2 appears as 10.10.4.1 and not as 10.10.2.2 (the address the probe actually reached). The client accepts it on its Link D interface (10.10.4.1 is directly connected there). *Hop 3:* the probe reaches the server, which answers ICMP Port Unreachable from 10.10.3.10; that reply also travels server→r2→client over Link D.

Server → client. *Hop 1:* r2 (server's gateway) drops the TTL=1 probe and replies out of LAN C, source 10.10.3.1. *Hop 2:* r2 forwards the TTL=2 probe straight over Link D to the client (`/32` route), and the client replies with Port Unreachable from the probed address 10.10.1.10. r1 never appears because it is not on the server→client path.

Why the directions differ. The routing is asymmetric on purpose (A2/A3): the client's only non-connected route is via r1, while r2's host route sends everything for the client over Link D. So the forward path has three hops and the reverse path two, and in the forward trace r2 shows up with its Link D address because its ICMP reply goes back over the reverse path.

SCREENSHOT TODO: `r2_traceroute_c2s.png`

On client: `traceroute -n 10.10.3.10`

Figure 5: `traceroute -n 10.10.3.10` on the client.

SCREENSHOT TODO: `r2_traceroute_s2c.png`

On server: `traceroute -n 10.10.1.10`

Figure 6: `traceroute -n 10.10.1.10` on the server.

R3. Reverse-path filtering

The value the kernel actually uses on an interface is max(`conf.all.rp_filter`, `conf.<if>.rp_filter`). All machines start with 1 everywhere and our scripts never touch `conf.all`, so it stays 1. Setting an interface to 0 would therefore have no effect ($\max(1, 0) = 1$); the only way to relax one interface is loose mode (2), which accepts a packet if its source is reachable via *any* interface. Strict mode (1) requires that the best route back to the source leaves through the interface the packet arrived on.

Machine	Interface	Value	Reason
client	IF_A	1	Packets on LAN A come from LAN A or via r1; their sources route back via IF_A.
client	IF_D	2	Server/r2 replies to 10.10.1.10 arrive here (A3) with sources like 10.10.3.10, but the client's route to 10.10.3.10 is the default via IF_A, so strict mode would drop every reply. Loose accepts them (the default route covers the source).
r1	IF_A, IF_B	1	Required by A7. It works: IF_A only sees LAN A sources, IF_B sees sources in LAN B/LAN C, which route back via IF_B (default). This is also why the client must use 10.10.1.10, not 10.10.4.2, as its source towards the server.
r2	IF_B	2	Client→server packets arrive here from r1 with source 10.10.1.10, but r2's best route to 10.10.1.10 is the `/32` via IF_D. Strict would drop the entire forward direction.
r2	IF_C, IF_D	1	IF_C only sees LAN C sources; IF_D only sees 10.10.4.2 (connected). The client never sends anything else over Link D.
server	IF_C	1	Required by A7. Single interface, every source routes back through it.
all	`conf.all`	1	Left strict, so effective values are exactly the ones above.

Table 2: rp_filter settings. Loose mode is used only where the asymmetric routing requires it.

SCREENSHOT TODO: `r3_rpfilter.png`

On each of client, r1, r2, server (one terminal per VM, or one after another over SSH): `sysctl -a 2>/dev/null | grep '\.rp_filter'`

Figure 7: Effective rp_filter settings on all four machines.

R4. Our client against a standard web server

On the server VM we serve a directory containing `index.html` and a random binary `bin300k.bin` (300 000 bytes, ≥100 KB) with `python3 -m http.server 8000`. On the client we fetch both files with our client and print the exit code, then fetch the same URLs with `curl` and compare the `md5sum`s. Both downloads must exit with 0 and the checksums must match pairwise.

SCREENSHOT TODO: `r4_html.png`

Server: `cd ~/r4 && python3 -m http.server 8000`.
Client: `sudo ./client/run-client http://10.10.3.10:8000/index.html -o a.html; echo $?`
`curl -s -o a_curl.html http://10.10.3.10:8000/index.html; md5sum a.html a_curl.html`

Figure 8: Our client downloading an HTML page from `python3 -m http.server`; exit code and md5sum vs curl.

SCREENSHOT TODO: `r4_binary.png`

Client: `sudo ./client/run-client http://10.10.3.10:8000/bin300k.bin -o b.bin; echo $?`
`curl -s -o b_curl.bin http://10.10.3.10:8000/bin300k.bin; md5sum b.bin b_curl.bin; ls -l b.bin`

Figure 9: Our client downloading a 300 KB binary; exit code and md5sum vs curl.

R5. curl, wget and our client against our server

On the server VM we run `sudo ./server/run-server -p 8080 -d ~/www` with `bin300k.bin` in the docroot and print its md5sum there. On the client we download the same file with `curl`, `wget` and our client (one after another, since our server handles one connection at a time) and compare all three checksums with the original.

SCREENSHOT TODO: `r5_three_clients.png`

Server: `md5sum ~/www/bin300k.bin; sudo ./server/run-server -p 8080 -d ~/www`
Client: `curl -s -o c.bin http://10.10.3.10:8080/bin300k.bin; echo $?`
`wget -q -O w.bin http://10.10.3.10:8080/bin300k.bin; echo $?`
`sudo ./client/run-client http://10.10.3.10:8080/bin300k.bin -o o.bin; echo $?`
`md5sum c.bin w.bin o.bin` (show the server-side md5sum in the same screenshot, e.g. split terminal)

Figure 10: curl, wget and our client downloading the same file from our server; all md5sums equal the original.

Supplementary (namespace testbed, not a substitute for the VM screenshots). We also ran an automated end-to-end suite (`test/e2e.sh`) on a network-namespace copy of the same topology built with our own `netcfg` scripts (in a Docker container). It covers our client against `python3 -m http.server` and other standard servers (HTML, 300 KB and 5 MB binaries, chunked encoding, redirects, error statuses), our server against curl, wget and our client (status codes 400/403/404/501, HEAD, content types, 300 KB/5 MB binaries), and transfers under `netem` impairments (5 % and 15 % loss, 25 % reordering, 5 % duplication, 2 % corruption, and all combined), plus checks of the packet logs. In the last recorded run 92 of 93 tests passed; the one failure is the $N = 3$ fairness case discussed in R6 (it exceeds our own 2× threshold).

R6. Packet capture and TCP mechanisms

SCREENSHOT TODO: `r6_capture.png`

Capture on the server, where both directions pass one interface (on the client, outgoing packets use IF_A and incoming ones IF_D).
Server: `sudo ./server/run-server -p 8080 -d ~/www` and in a second terminal `sudo tcpdump -nn -i <IF_C> 'tcp port 8080'`
Client: `sudo ./client/run-client http://10.10.3.10:8080/index.html -o x.html`
(Use a small file so SYN/SYN-ACK/ACK, request, response and both FINs fit on one screen.)

Figure 11: `tcpdump -nn` capture of one complete connection of our client (handshake, request, data, ACKs, FIN exchange).

All of this is in `src/tcp.py`. Sequence numbers are compared modulo 2^{32}.

Retransmission. We keep one retransmission timer for the oldest unacknowledged byte (`snd_una`). RTT is sampled on one segment at a time and smoothed as in RFC 6298 ($srtt \leftarrow \frac{7}{8}srtt + \frac{1}{8}r$, $rttvar \leftarrow \frac{3}{4}rttvar + \frac{1}{4}|srtt - r|$), with $RTO = \min(srtt + \max(4 \cdot rttvar,\ 200\text{ ms}),\ 1\text{ s})$; the initial RTO is 1 s. Karn's algorithm: the pending RTT sample is discarded whenever anything is retransmitted, so retransmitted segments are never timed. When the timer expires we double the RTO (still capped at 1 s), set `snd_nxt` back to `snd_una` and resend the first segment; the rest is then resent go-back-N style as the window opens. If data is outstanding and nothing new has been acknowledged for 10 s, the connection is aborted. Fast retransmit: on the third duplicate ACK the first unacknowledged segment is resent immediately. Before that, the first two duplicate ACKs each allow one extra new segment beyond `cwnd` (limited transmit, RFC 3042), so a small window can still produce three dup ACKs. SYN/SYN-ACK are retried every 1 s for 8 s, and our FIN is retransmitted until ACKed, giving up after 2 s.

Out-of-order buffering. In-order data is appended to the receive buffer and `rcv_nxt` advances. A segment that starts above `rcv_nxt` but within our 65 535-byte advertised window is stored in a dictionary keyed by its sequence number (keeping the longer copy on duplicates). Bytes below `rcv_nxt` are trimmed, so duplicates are delivered only once. Whenever the gap at `rcv_nxt` is filled, we repeatedly drain every buffered segment that now starts at or below `rcv_nxt`, trimming overlap, until no more progress is made. A peer FIN is only consumed once `rcv_nxt` reaches its sequence number. We send a cumulative ACK for every data segment, including duplicates and out-of-order ones (no delayed ACKs). These duplicate ACKs are what drive the sender's fast retransmit.

Congestion control. Reno, with NewReno handling of partial ACKs. Data in flight never exceeds $\min(cwnd,\ \text{peer window})$.

- *Slow start:* initial window 10 MSS; while $cwnd < ssthresh$, each new ACK adds min(acked, MSS).
- *Congestion avoidance:* each new ACK adds $\text{MSS}^2/cwnd$ (about 1 MSS per RTT).
- *Fast recovery:* on the 3rd dup ACK, $ssthresh = \max(\text{flight}/2,\ 2\,\text{MSS})$, $cwnd = ssthresh + 3\,\text{MSS}$ and `recover` is set to the highest sequence sent. Each further dup ACK inflates `cwnd` by 1 MSS. A partial ACK (below `recover`) immediately retransmits the next hole and deflates `cwnd` by the amount acked (NewReno); a full ACK ends recovery with $cwnd = ssthresh$. A new fast retransmit is not started until `snd_una` passes `recover`.
- *Timeout:* $ssthresh = \max(\text{flight}/2,\ 2\,\text{MSS})$, $cwnd = 1\,\text{MSS}$, recovery is cancelled, then slow start again.

Fairness measurements (namespace testbed). We reproduced the bottleneck described on Piazza: 10 Mbit/s with 10 ms delay and a 30-packet (45 000 B) FIFO on r2's Link D egress, plus 10 ms delay on r1→r2 (netem + tbf, with TSO/GSO/GRO off). N of our client←our-server flows run at the same time as N curl←`python3 -m http.server` flows (kernel TCP sender, CUBIC), downloading an 80 MB file. Throughput is counted at the client over a 20 s window after 5 s of warm-up. Jain's index is over all $2N$ flows.

N	fair share	ours, mean (Mbit/s)	curl, mean (Mbit/s)	ratio ours/curl	Jain
1	4.96	5.73	4.17	1.37	0.976
2	2.48	3.28 (4.03, 2.53)	1.67 (1.59, 1.75)	1.96	0.868
3	1.65	2.27 (2.32, 2.37, 2.11)	1.04 (0.88, 1.07, 1.16)	2.18	0.875

Table 3: Last run (`test/results/e_fairness_N*/flows.txt`); link total 9.91 Mbit/s in every case. Over all recorded runs the ratio was 1.33–1.38 for N=1, 1.70–2.47 for N=2 and 2.18–2.81 for N=3.

The link is always fully used and no flow starves. With a single flow of each kind the split is reasonably fair, but our flows get more than their share as N grows (about 2× the curl flows at $N = 3$). We think the cause is where the drops land, not a more aggressive window rule. Our receiver ACKs every segment, so our sender is clocked out one segment per ACK and its packets spread over the RTT. curl's receiver (the Linux stack) uses delayed ACKs. Each of its ACKs covers two or more segments, so the kernel sender emits back-to-back bursts, and bursts are much more likely to overflow a 30-packet tail-drop queue. At $N = 3$ we measured about 1.8 % of our segments lost/retransmitted against about 7 % for the curl flows (the qdisc dropped 1145 of 37 181 packets overall). For loss-based TCP, throughput $\propto \frac{\mathrm{MSS}}{RTT\sqrt{p}}$, so $\sqrt{7/1.8} \approx 2$, which matches the observed ratio. Per-ACK growth also helps us a little: with one ACK per segment, Reno's $\mathrm{MSS}^2/cwnd$ rule gives the full 1 MSS per RTT. So our control law is standard Reno, but the bottleneck penalises the burstier kernel flows more.